



streaky

COMPLETE CODEBASE & ARCHITECTURE ANALYSIS

A comprehensive technical review detailing the directory hierarchy, file utilities, data flows, state management systems, and dead-weight analysis of the Streaky Flutter application.

PROJECT CODENAME: STREAKY (MOBILE & WEB)
CORE FRAMEWORK: FLUTTER 3.12+ / DART SDK
DATA PERSISTENCE: CLOUD FIRESTORE & LOCAL PREFERENCES
DOCUMENT VERSION: 1.0 (PRODUCTION AUDIT)
AUDITED ON: JUNE 2026

Table of Contents

- Chapter 1: Executive Summary & Overview: An introduction to the Streaky app, its primary goals, and the integrated core technologies.
- Chapter 2: Codebase Directory Walkthrough: A detailed, file-by-file analysis of the configuration files, data models, state providers, database services, UI widgets, and screen directories.
- Chapter 3: Core Architectural Systems: Deep dive into the data pipelines, user authentication flow, real-time sync mechanism (including the complex weekday and weekly streak reset formulas), notification engine, and the gamified level systems.
- Chapter 4: Cleanup & Optimizations (Redundant Files): A formal audit listing non-essential developer scripts, duplicates, and empty configurations that can be safely deleted to reduce bundle size by 2.24 MB and clean up repo structures.

Chapter 1: Executive Summary & Overview

Streaky is a modern, gamified habit-tracking application designed to help users build and sustain habits through interactive daily streaks, animated mascot feedback, real-time reminders, and statistics. The project follows a clean architecture model implemented in Flutter, utilizing Google's Cloud Firestore for real-time data storage, Firebase Authentication for session management, local notifications for scheduling time-zone aware reminders, and Provider for state propagation.

The application is structured to decouple UI presentation from business logic. State flows top-down from Providers, which listen to active Firestore data streams, and propagates reactive changes directly to the screens. This architectural pattern ensures responsiveness, supports offline synchronization, and optimizes widget rebuild cycles.

Core Technology Stack Details

- SDK & Framework: Flutter (Dart 3.12+ SDK), structured for cross-platform compatibility across Android, iOS, Web, and Windows.
- State Management: Provider (v6.1.2) - lightweight, reactive state management using ChangeNotifier to broadcast model changes.
- Navigation & Routing: GoRouter (v14.8.1) - declarative, URL-based router facilitating nested shell navigation (main bottom tabs) and custom deep links.
- Data Storage & Backend: Firebase Core (v3.13.0) and Cloud Firestore (v5.6.7) - handles all remote CRUD processes, user account records, and structured streak logs.
- Local Notifications: flutter_local_notifications (v22.0.0-dev.3) & timezone (v0.11.0) - handles background scheduling of timezone-accurate push alerts.
- Visual Analytics: fl_chart (v0.70.2) - compiles complex streak heatmaps, monthly activity circles, and weekly progress statistics into rich UI charts.
- Animations: flutter_animate (v4.5.2) - adds fluid micro-animations (mascot states, count increments, glow cards) to improve user engagement.

Chapter 2: Codebase Directory Walkthrough

This chapter maps the physical files and folders within the 'lib/' directory, explaining their direct function, design parameters, and importance in the execution of the application.

1. Application Configuration (lib/config/)

The configuration files house constant data definitions, navigation parameters, global theme configurations, color palettes, and database keys. This ensures uniformity and eliminates hardcoded values across screens.

lib/config/app_colors.dart

CONFIG

Declares the centralized color palette including primary colors (Orange, Purple), surface states, glowing cards, and custom HSL colors used to style the dark-theme UI.

lib/config/app_text_styles.dart

CONFIG

Defines consistent Google Font text schemas (Outfit for body text, Fredoka for bold headings) with pre-configured weights, tracking, and font sizes.

lib/config/app_theme.dart

CONFIG

Builds the application's global dark ThemeMode, tailoring inputs, buttons, sliders, text fields, and shell borders to match glassmorphic layouts.

lib/config/app_router.dart

CONFIG

Initializes GoRouter navigation. Maps paths to screens (e.g. /splash, /auth, /reminders, /streak-details) and builds the ShellRoute for persistent bottom navigation tabs.

lib/config/motivational_quotes.dart

CONFIG

Contains an array of motivational quote objects. Dynamically retrieved and displayed on the Home Dashboard header to inspire users.

lib/config/streak_categories.dart

CONFIG

Defines default category types (Health, Mind, Work, Fitness, Custom), mapping each category to a specific color palette, icon, and default emoji.

2. Data Schemas & Models (lib/models/)

Data models formalize the exchange of data between the UI, State Providers, and Firebase Firestore. Each model declares serialization (toMap) and deserialization (fromFirestore/fromMap) methods to interface with database collections.

lib/models/user_model.dart

[CORE FILE](#)

Declares schema details for users: UID, email, name, avatar, bestStreak, totalXP, currentLevel, and lastActive timestamp. Governs level calculations.

lib/models/streak_model.dart

[CORE FILE](#)

Declares streak items: ID, title, emoji, category, color, frequency (daily, weekdays, weekly), reminderTime, currentStreak, bestStreak, and position index.

lib/models/checkin_model.dart

[CORE FILE](#)

Represents a single check-in transaction: date string (YYYY-MM-DD), completed status, completedAt timestamp, checkin method, and mood value.

lib/models/achievement_model.dart

[CORE FILE](#)

Models custom gamified achievements (badges). Evaluates user unlock requirements (total XP, check-in count, category count) and determines progress percentages.

3. State Providers (lib/providers/)

Providers operate on the ChangeNotifications pattern, loading data from database streams and notifying listener widgets to rebuild dynamically.

lib/providers/auth_provider.dart

[CORE FILE](#)

Tracks the current active Firebase User, manages processing flags during login/sign-up, and propagates authentication changes app-wide.

lib/providers/user_provider.dart

CORE FILE

Synchronizes user levels and XP. Increments XP on check-ins, calculates progress fractions, and manages profile/avatar updates.

lib/providers/streak_provider.dart

CORE FILE

The largest class in the app. Manages lists of active/archived/completed streaks, handles check-ins, performs calendar reordering, and triggers state synchronization.

lib/providers/notification_provider.dart

CORE FILE

Manages historical notifications, load indicators, and notification toggle configurations.

4. Services Layer (lib/services/)

Services encapsulate low-level external API integration (Authentication, Firebase SDKs, Local Device Notifications) and isolate these side-effects from state providers.

lib/services/auth_service.dart

CORE FILE

Wraps Firebase Auth API calls for Email Sign-Up/Sign-In, Google Sign-In (using google_sign_in package), and global Sign-Out.

lib/services/firestore_service.dart

CORE FILE

Performs all CRUD queries on Cloud Firestore. Handles subcollections: users/{userId}/streaks/{streakId}/checkins/{checkinId}. Safely handles renaming and deletions.

lib/services/notification_service.dart

CORE FILE

Configures flutter_local_notifications. Handles timezone registrations, schedules exact alarms, handles notification clicks, and manages active alarm syncing.

5. Shared UI Widgets (lib/widgets/)

Shared widgets compile repetitive UI components to enforce consistent styling and simplify screen templates. They utilize animations and theme properties directly.

lib/widgets/animated_counter.dart

CORE FILE

Displays changes in numerical scores (like streak days or user XP) using scroll-wheel animations to emphasize score increases.

lib/widgets/fire_mascot.dart

CORE FILE

Renders the animated mascot (fire_mascot.gif) with contextual responses based on daily habit progress (happy, sleeping, motivated).

lib/widgets/glassmorphic_container.dart

CORE FILE

Renders a transparent glass-like layout with gradient borders and blur filters to implement the premium design theme.

lib/widgets/glow_card.dart

CORE FILE

Combines custom container borders with glowing drop-shadows to highlight active widgets, widgets that have achievements, or buttons.

lib/widgets/particle_background.dart

CORE FILE

Animates floating particle circles in the background of screens (e.g. login, splash) using canvas painting to create dynamic motion.

lib/widgets/streak_check_button.dart

CORE FILE

Custom high-performance interactive check-in button displaying tap-animations and success states.

6. Screens & View Layers (lib/screens/)

The screens represent complete full-page view layouts mapped to the router. Most screens contain a subfolder 'widgets/' to store widgets localized exclusively to that screen.

- `lib/screens/main_shell.dart`: Main tab container. Controls bottom navigation animations, routes, and safely closes active sheets before tab transitions.
- `lib/screens/splash/`: Splash screen displaying app branding (`app_logo.gif`) and particle effects while resolving automatic session login.
- `lib/screens/auth/`: Full login, registration, and password retrieval template with integrated error alerts and validation indicators.
- `lib/screens/home/`: Home Screen (Today tab) containing greetings, motivational quotes, progress gauges, and lists of pending or completed habits.
- `lib/screens/home/widgets/`: Local dashboard items: greeting headers, dynamic quote boxes, weekly dots (seven weekdays), and summary cards.
- `lib/screens/stats/`: Stats Dashboard showing consistency meters, monthly activity heatmaps, and weekly check-in charts using the `fl_chart` parser.
- `lib/screens/stats/widgets/`: Analytics components: consistency circles, monthly heatmaps (grid of active dates), and weekly bar charts.
- `lib/screens/streaks/`: Main Streaks tab displaying current habits, enabling categories filtering, and drag-and-drop manual list reordering.
- `lib/screens/streaks/widgets/`: Streak details components: category chips, streak cards, and the Add Streak Sheet drawer.
- `lib/screens/rewards/`: Rewards Screen displaying achievements grid, unlocked badges, mascot evolution states, and current level progress bars.
- `lib/screens/profile/`: Profile settings enabling avatar changes, name updates, password reset emails, and sign-out triggers.
- `lib/screens/reminders/`: Reminders screen allowing customization of notification tones, intervals, and scheduling rules.

Chapter 3: Core Architectural Systems

Understanding how individual components communicate is critical to maintaining the Streaky application. The application relies on three core data-flow systems to operate.

1. Reactive State Propagation & Firebase Pipelines

When the application launches, the AuthProvider checks the active login session. Once resolved, the UserProvider and StreakProvider are initialized using the User's UID. Rather than calling Firestore once (which would miss updates made on other devices), the providers listen to real-time streams:

- UserProvider listens to: `users/{userId}`
- StreakProvider listens to: `users/{userId}/streaks`

When Firestore updates, changes are automatically pushed to the streams, which trigger `notifyListeners()` inside the provider, updating the UI immediately.

2. Daily & Weekly Streak Synchronization Logic

On application startup (or when receiving new database updates), the StreakProvider runs a synchronization routine (`_syncStreakStates`) to evaluate if active streaks have been broken. This logic handles three distinct frequency configurations:

- **Daily Habits:** Checks if `lastCheckIn` occurred today or yesterday. If more than 1 day has passed, it resets `currentStreak` to 0 and sets `completedToday` to false.
- **Weekday-Only Habits:** If the current day is Saturday or Sunday, check-in is excused: `completedToday` is marked true. If it is a weekday: checks if `lastCheckIn` is today. If it is Monday, it allows the previous check-in to be on Friday (excusing the weekend). If it is Tuesday through Friday, the previous check-in must be yesterday; otherwise, the streak resets to 0.
- **Weekly Habits:** Finds the start of the current week (Monday). If the last check-in occurred within the same week, `completedToday` is set to true. If it occurred in the previous week, `completedToday` is false, but the streak continues. If more than 7 days have passed, `currentStreak` resets to 0.

3. Timezone-Aware Notification Engine

Local notification scheduling is handled in a background pipeline to keep reminders accurate:

1. NotificationService initializes the local location database (using timezone data) and resolves timezone differences (e.g. mapping Calcutta to Kolkata).
2. On start, the StreakProvider calls `_syncNotifications`, which cancels scheduled alarms for archived or deleted streaks.
3. For active, enabled reminders, it schedules a daily recurring alarm using `zonedSchedule` with exact timing (`exactAllowWhileIdle`).
4. If exact alarms are not permitted by the user's OS settings, the service catches the exception and schedules an inexact alarm (`inexactAllowWhileIdle`) as a fallback.

4. Gamification & XP System

The gamification cycle is triggered directly by check-ins:

- Completing a habit awards the user +10 XP (`xpPerCheckin`).
- Maintaining streak milestones awards bonuses: +50 XP (7 days), +100 XP (14 days), +250 XP (30 days).
- The UserProvider updates Firestore with the new XP value. The level is calculated dynamically using a loop: each level requires $(\text{level} * 100)$ XP. Accumulating enough XP automatically increments `currentLevel`, unlocking new badges.

Chapter 4: Cleanup & Optimizations (Redundant Files)

A comprehensive audit identified several files in the workspace that are redundant, unused, or duplicate. These files can be safely deleted. They do not affect the build or runtime of the app.

process_gif.py A developer utility script used to remove black backgrounds from the logo GIF and export static PNG frames. Since the final assets (app_logo.gif and app_logo.png) are already saved in assets/images/, this script is no longer used by the application.	REMOVABLE
Requirement.md An empty markdown file in the project's root folder. It contains no data and is not referenced by any source files.	REMOVABLE
assets/images/app_logo.png A 1.12 MB PNG image file generated by the process_gif script. The application exclusively uses 'app_logo.gif' for splash and login screens. This file is never referenced in Dart code.	REMOVABLE

assets/images/app_logo_processed.png

REMOVABLE

A duplicate copy of 'app_logo.png' (1.12 MB) created during developer testing. It is completely redundant and never referenced by the application.

streaky.iml & .idea/

REMOVABLE

Configuration files for IntelliJ IDEA and Android Studio. If developers are using VS Code or running command-line builds, these files are not utilized and add clutter to the repository.

Direct Benefits of Removing Redundant Files

1. Reducing App Bundle Bloat:

In 'pubspec.yaml', the asset folder is declared as a folder-level wildcard:

assets:

- assets/images/

This means Flutter bundles EVERY file inside that folder into the final binary (APK/IPA). The two unused images ('app_logo.png' and 'app_logo_processed.png') are 1.12 MB each. Deleting them removes 2.24 MB of dead weight directly, reducing the download size for end-users.

2. Clean Repository and Cleaner Code:

Removing helper scripts ('process_gif.py') and empty logs ('Requirement.md') reduces noise in the file tree, making code audits faster and decreasing git repository sizes.

3. Faster Build Speeds:

Fewer assets mean faster asset-bundling stages during 'flutter build' operations.

RECOMMENDED ACTION PLAN:

1. Delete process_gif.py and Requirement.md.
2. Delete assets/images/app_logo.png and app_logo_processed.png.